다듬 질의응답 대비서

파트 1. 정체성·목적
파트 2. 사용법·화면
파트 3. 전체 구조
파트 4. 검사기 (규칙)
파트 5. 모델·데이터
파트 6. 평가·수치
파트 7. 배포·보안·라이선스
파트 8. 개발 과정·팀
파트 9. 곤란한 질문
파트 10. 숫자 치트시트 (1장)

다듬 질의응답 대비서

이 문서만 보고 질문에 답할 수 있게 만들었습니다. 질문마다 **한 줄 답**이 먼저 나오고, 더 필요하면 아래 근거를 봅니다.

읽는 법: 🟢 한 줄 답 · 📎 근거(어디서 확인했나) · ⚠️ 말조심(틀리기 쉬운 부분)

파트	내용	질문
1	정체성·목적	Q1-Q14
2	사용법·화면	Q15-Q28
3	전체 구조	Q29-Q42
4	검사기(규칙)	Q43-Q74
5	모델·데이터	Q75-Q100
6	평가·수치	Q101-Q124
7	배포·보안·라이선스	Q125-Q142
8	개발 과정·팀	Q143-Q156
9	곤란한 질문	Q157-Q176
10	숫자 치트시트	—

파트 1. 정체성·목적

Q1. 다듬이 뭔가요?

🟢 한국어 **맞춤법·띄어쓰기·조사·문법**을 교정해 주는 도구입니다. 틀린 곳에 밑줄이 그어지고 클릭 한 번에 고칩니다. 📎 책 0장, README

Q2. 한 문장으로 특징을?

🟢 **"틀리게 고치는 것을 가장 무겁게 보는, 완전 로컬 교정기."** 📎 책 13장 원리 1.1

Q3. 왜 만들었나요?

🟢 기존 교정기는 **멀쩡한 글을 틀리게 고치는 일**이 있어, 한 번 겪은 사용자는 도구를 끈다는 문제의식입니다. 📎 DESIGN.md 1.1: "틀리게 고침 → '얘 뭐야' → 한 번이면 끈다"

Q4. 어디서 쓸 수 있나요?

🟢 **웹 앱 · 크롬 확장 · macOS 앱** 세 가지입니다. 셋 다 같은 엔진을 씁니다. 📎 책 12장

Q5. 어떤 오류를 잡나요?

🟢 다섯 종류: **맞춤법 · 띄어쓰기 · 조사 · 문법 · 문체**. 📎 `schemas.py`의 `Category` = spacing / spelling / josa / grammar / style

Q6. 외부 AI(GPT 등)를 쓰나요?

🟢 **아니요.** 외부 API를 부르지 않고, 모델은 이 저장소에서 처음부터 학습한 작은 모델들입니다. 📎 NOTICE: "no external AI service is called at build or run time" ⚠️ 초기(7월)에는 Gemini 3층이 있었으나 9/15에 **제거**했습니다.

Q7. 글이 외부로 나가나요?

🟢 **안 나갑니다.** 서버가 내 컴퓨터(`localhost`)에서 돌고, 확장은 `localhost:8000`으로만 보낼 수 있게 묶여 있습니다. 📎 SECURITY.md

Q8. 회원가입·결제가 있나요?

🟢 **없습니다.** 7월에 만들었던 계정·결제 기능은 9/15에 전부 제거했습니다. 📎 커밋 `feat!: remove billing, accounts, and the service layer`

Q9. 이름 "다듬"의 뜻은?

🟢 "글을 다듬다". 슬로건은 **"거친 생각에서, 단단한 문장으로."** 모델군 이름 "결(Gyeol)"은 나뭇결을 따라 깎는다는 뜻입니다. 📎 README, `linear_model.py` 설명

Q10. 누가 만들었나요?

🟢 저장소 소유자(zmflm0110)가 7/14~9/17에 253커밋, 다른 계정(byeolki)이 9/15~9/21에 124커밋을 했습니다. 📎 `git log` 집계 ⚠️ zmflm0110 커밋 253개 중 239개에 `Co-Authored-By: Claude`가 붙어 있습니다(Claude Code 세션으로 작성).

Q11. 라이선스는?

🟢 코드는 **MIT**입니다. 데이터(위키백과 CC BY-SA 4.0, nsmc CC0)는 각자의 라이선스를 따릅니다. 📎 LICENSE, NOTICE

Q12. 오픈소스인가요?

🟢 코드 라이선스는 MIT(오픈소스 라이선스)입니다. 단, 제가 확인한 시점에 `dadeum-book` 책 저장소는 **비공개**입니다. 📎 `gh repo view`로 확인(책 저장소). 다듬 본 저장소의 공개 여부는 확인하지 않았습니다.

Q13. 한국어만 되나요?

🟢 네. 한글이 들어간 글만 검사합니다(확장은 한글이 없으면 건너뜀). 📎 `content.js`의 `HAS_KOREAN`

Q14. Grammarly나 다른 한국어 교정기와 뭐가 다른가요?

🟢 ① 완전 로컬, ② **오탐 최소화를 설계 원칙으로**, ③ 성적(특히 처음 보는 문장)을 정직하게 공개. ⚠️ 다른 제품의 성능과 직접 비교한 데이터는 이 자료에 **없습니다**. 비교 수치를 말하지 마세요. 📎 DESIGN.md 1.x, 책 11장

파트 2. 사용법·화면

Q15. 어떻게 실행하나요?

🟢 서버를 띄우면 됩니다.

```
cd backend && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000      # http://localhost:8000
```

📎 README “실행 방법”

Q16. 맥 앱은 어떻게 설치하나요?

🟢 `sh desktop/make-app.sh` → `/Applications/DaDeum.app` 설치, 첫 실행 때 **손쉬운 사용 권한**을 켭니다. 📎 README

Q17. 밑줄 색은 무슨 뜻?

🟢 **카테고리**입니다(맞춤법·띄어쓰기·조사·문법·문체). 테마(밝음/어두움)와 상관없이 색이 카테고리를 뜻합니다. 📎 README

Q18. 실선과 점선의 차이?

🟢 **실선** = 확실(high/medium), 클릭하면 바로 고침. **점선** = 애매(low), 설명만 하고 자동으로 안 고침. 📎 책 5장, `isActionable()`

Q19. 카드에는 뭐가 나오나요?

🟢 **무엇을 → 무엇으로, 왜**(규정 조항·반례까지). 버튼은 **교정 · 무시 · 이 표현은 항상 허용** 세 갈래. 📎 README “카드는 ’고쳐라’가 아니라 ’왜’를 말합니다”

Q20. 한 번에 다 고칠 수 있나요?

🟢 **“모두 고치기”**가 실선만 앞에서부터 차례로 고칩니다. 점선은 건드리지 않습니다. 📎 책 12장 `applyAll`

Q21. 실수로 고쳤으면?

🟢 **되돌리기(undo)**, 최대 50단계. 모두 고치기는 한 번에 묶여 되돌려집니다. 📎 `MAX_UNDO = 50`

Q22. 틀리지 않았는데 지적하면?

🟢 **무시**(그 교정만) 또는 **항상 허용**(그 단어를 개인 사전에). 신고하면 그 문장이 회귀 테스트가 됩니다. 📎 책 12장, README "함께 만들기"

Q23. "항상 허용"은 어디에 저장되나요?

🟢 **브라우저/앱의 로컬 저장소**뿐입니다. 서버로 안 갑니다. 📎 SECURITY.md

Q24. 키보드 단축키는?

🟢 `Tab` 차례로 고치기, `F3` 하나씩 넘겨보기(웹), `^⌥D` 맞춤법 검사 창(맥). 📎 README

Q25. 얼마나 긴 글까지?

🟢 서버는 **2만 자**, 웹 화면은 현재 **5천 자**까지입니다. 서버는 4천 자씩 문장 경계에서 잘라 돌립니다. 📎 `CheckRequest max_length`, `MAX_TEXT_LEN`, `CHUNK_LIMIT`

Q26. 타이핑하면 바로 검사하나요?

🟢 네. 단어·문장이 끝나면 0.25초, 멈추면 0.7초, 어떤 경우에도 **최대 0.9초** 안에 한 번 검사합니다. 📎 `AUTO_CHECK_*`

Q27. 한글 입력 중에 방해되지 않나요?

🟢 **조합 중에는 아무것도 안 건드립니다**(compositionstart~end). 📎 `content.js` 맨 위 "Critical Korean-specific rule"

Q28. 사파리·모바일은?

🟢 사파리는 **맥 앱이 담당**합니다(다듬 확장이 없으므로 맥 앱이 비켜서지 않음). 모바일은 **미착수**입니다. 📎 `extensionCoveredBrowsers`, PLAN-system-wide-assistant 로드맵

파트 3. 전체 구조

Q29. 전체 구조를 한 그림으로?

🟢

```
화면(웹·확장·맥) —POST /api/check→ 서버
   서버: ① 형태소 분석 → ② 검사기 21개가 후보 제출 → ③ 겹치는 후보 정리
화면 ←— issues[] —— 밑줄·카드·고치기 (공유 코어가 상태 관리)
```

📎 책 0·5장

Q30. 서버와 화면을 왜 나눴나요?

🟢 교정 로직을 **한 곳**에 두고 화면 세 개가 같은 서버를 쓰게 하려고요. 로직이 흩어지면 어긋납니다. 📎 책 4장

Q31. 엔진은 무슨 언어/틀?

🟢 **Python + FastAPI**. 의존성은 6개(fastapi, uvicorn, gunicorn, kiwipiepy, pydantic, python-dotenv). 📎 README

Q32. 형태소 분석은 직접 만들었나요?

🟢 **아니요, Kiwi(kiwipiepy)라는 외부 라이브러리**입니다. 규칙 검사기 대부분이 Kiwi 분석 위에서 돕니다. ⚠️ "전부 직접 만들었다"고 말하면 안 됩니다. **직접 학습한 것은 결(Gyeol) 모델군**입니다. 📎 NOTICE에는 Kiwi가 적혀 있지 않습니다(보완 필요).

Q33. 검사기가 몇 개인가요?

🟢 엔진에 등록된 검사기(`GENERATORS`)가 **21개**, 지적의 출처(source)는 **58개**입니다. 📎 `engine.py`, `sources.py`

Q34. 검사기들은 어떻게 협력하나요?

🟢 서로 모릅니다. 각자 후보를 내고, **마지막에 한 번 병합**이 정리합니다. 📎 책 5장

Q35. 겹치면 누가 이기나요?

🟢 ① **고칠 말이 있는 쪽** ② 출처 우선순위(규칙 6 > 사전 5 > 모델 4 > …) ③ 같으면 먼저 나온 쪽. 📎 `merge_and_dedupe`

Q36. 왜 "제안 있는 쪽"이 먼저인가요?

🟢 이상탐지가 `자률신경` 전체를 잡아 그 안의 `률→율` 교정이 사라지는 사고가 있었기 때문입니다. 📎 DESIGN 1.4

Q37. "층"은 무슨 뜻?

🟢 분류 이름입니다. **규칙 / 모델 / 조언** 세 종류. 실행 순서가 아니라 구분입니다. 📎 `sources.py` layer

Q38. 한 층이 고장 나면?

🟢 **그 층만 조용히 빠지고 앱은 뜹니다.** 어느 층이 켜졌는지는 `/api/health`로 봅니다. 📎 DESIGN 1.5·1.6

Q39. 응답 시간은?

🟢 문서 기준 짧은 문장 약 2ms, 1,400자 약 90ms. **서버를 처음 띄운 직후 첫 요청만** 사전 적재로 수 초. 📎 DEPLOY.md, 제가 직접 측정한 첫 요청 4.3초(콜드 스타트)

Q40. "공유 코어"가 뭔가요?

🟢 밑줄·고치기·무시·되돌리기 **상태 관리를 한 번만 쓴** 코드(JS 원본 + Swift 포팅, 같은 테스트). 📎 책 12장

Q41. 왜 공유 코어를 만들었나요?

🟢 같은 로직이 세 화면에 **따로 구현돼 서로 어긋났기** 때문입니다. 📎 README "공유 클라이언트 코어"

Q42. 코어를 확장도 쓰나요?

🟢 **일부만** 씁니다(signature, isActionable, isIgnored). 교정 적용은 아직 자체 구현입니다. 📎 README 한계 표

파트 4. 검사기 (규칙)

Q43. 규칙 검사기는 어떻게 일하나요?

🟢 ① 형태소 태그·정규식으로 **의심 자리 찾기** → ② **함정(오탐 자리) 거르기** → ③ 지적 만들기. 코드의 80%는 ②입니다. 📎 책 6.0

Q44. 받침이 있는지 어떻게 아나요?

🟢 **산수**입니다. `(글자번호 − 44032) % 28`이 받침 번호이고 0이면 받침 없음. 📎 책 2장 `hangul.py`

Q45. 왜 28로 나누나요?

🟢 한글은 `초성19 × 중성21 × 종성28 = 11,172자`가 번호순으로 놓여 있고, 종성이 가장 안쪽 단위라 28로 나눈 나머지가 종성입니다. 📎 책 2.2

Q46. 조사 `을/를`은 어떻게 고르나요?

🟢 앞말 마지막 글자 받침이 **없으면 를/는/가/와/로, 있으면 을/은/이/과/으로**. `ㄹ` 받침은 `로`가 예외입니다. 📎 `josa.py`

Q47. 틀린 조사를 형태소 분석기가 못 찾지 않나요?

🟢 맞습니다. `학교을`의 `을`을 Kiwi는 **명사로 읽습니다**. 그래서 "조사 태그"가 아니라 **명사 바로 다음 글자**를 직접 봅니다. 📎 책 3.6

Q48. `가수이다`를 `가수가다`로 잘못 고치지 않나요?

🟢 안 고칩니다. 진짜 조사 뒤에는 공백·부호·끝이 와야 하는데 `이` 뒤에 `다`가 이어지므로 조사가 아니라고 봅니다. 📎 `_particle_boundary_ok`

Q49. `뭔가`·`건가`는요?

🟢 어미 `-ㄴ가`가 앞 음절에 **융합**된 것(종성 자모로 시작)이라 조사로 안 봅니다. 📎 `_ending_tail_positions`, 구어 정문 오탐 25건 중 17건이 이 부류

Q50. 숫자 뒤 조사(`3를`)는요?

🟢 **판단하지 않습니다.** "삼"으로 읽는지 "쓰리"로 읽는지 몰라서, 켜 보니 오탐이 3배가 됐기 때문입니다. 📎 `josa.py` 주석 (0.05% → 0.15%)

Q51. 사전 규칙은 몇 개고 어떻게 만드나요?

🟢 **474개** 정규식. 앞뒤 문맥을 넣어 구분합니다(`카드로 결재` → `결제`, `서류를 결재`는 그대로). 📎 `dict_rules.json`

Q52. 사전 규칙이 틀린 적은?

🟢 있습니다. **정문을 고치려던 규칙 9개**를 찾았습니다(예: 인물 `성희안`을 `성희한`으로). 경계 조건을 달거나 low로 내렸습니다. 📎 DESIGN 1.10-c

Q53. 율/률은?

🟢 앞 글자가 **모음이나 ㄴ 받침이면** `율`, 아니면 `률`. 단어 끝은 접미사 태그로, 단어 속(`자률신경`)은 언어모델 점수로 확인합니다. 📎 책 6.3

Q54. 의존명사 띄어쓰기는?

🟢 **관형사형 어미(ETM) 뒤의 의존명사**(것·수·뿐·줄…)만 확정해 띄웁니다. `먹을것` → `먹을 것`. 📎 책 6.5

Q55. `-ㄹ걸`, `-는데`, `갈릴리`는 왜 안 고치나요?

🟢 각각 종결어미 / 연결어미 / 지명이기 때문입니다. 단어마다 **틀(frame)을 확인**하고 맞을 때만 고칩니다. 📎 `dependent_noun.py`

Q56. 간접 인용 `온다라고`는?

🟢 종결어미 바로 뒤에 `라고`가 붙으면 겹침이라 `온다고`로. 사이에 따옴표가 있으면(직접 인용) 건드리지 않습니다. 📎 `quotation.py`

Q57. 물음표는 어떻게 아나요?

🟢 **의문 어미 + 같은 문장 안의 의문사**(어디·몇·왜…)가 둘 다 있을 때만 `.`→`?`. 그래서 `나야.`는 그대로입니다. 📎 `question_mark.py`

Q58. 사물 존대(`커피 나오셨습니다`)는요?

🟢 **사물 목록**(animacy.json)에 있는 명사가 주어일 때만 고칩니다. 사람인지 사물인지는 어떤 태그에도 없어서 목록을 씁니다. 📎 `honorific.py`

Q59. `에게/에` 구분은?

🟢 유정물/무정물 **명사 목록** + `에`는 **수여 동사가 있는 문장일 때만**. `에`의 다른 쓰임(장소 등)이 너무 많아서 교정 쪽을 허가제로 했습니다. 📎 `animacy_particle.py`

Q60. 시제 호응은?

🟢 시간 부사(어제·내일…)와 서술어 시제가 **정면 모순일 때만**(과거 부사 + 현재형 등). 역사적 현재는 건드리지 않습니다. 📎 `tense.py`

Q61. `전혀 다르다`는 왜 안 잡나요?

🟢 `다르다·새롭다` 같은 **허용 서술어**가 있으면 정문입니다. 부정 호응 규칙의 예외 목록입니다. 📎 `negative_polarity.py`

Q62. `-시키다`는 항상 틀린 건가요?

🟢 아닙니다. `교육시키다`처럼 `-하다`가 이미 타동사인 **명사 목록**일 때만 군더더기로 봅니다(`변화시키다`는 필요). 📎 `redundant_causative.py`

Q63. 이중 피동은 어떻게 잡나요?

🟢 **형태소 배열**을 봅니다: 피동사 어간 + `-어` + `지(VX)`. 어간을 고치고 `kiwi.join`으로 활용형을 되살려 `잊혀졌다→잊혔다`. 📎 책 6.15

Q64. 왜 문자열 규칙 대신 형태소 규칙인가요?

🟢 문자열 규칙은 활용형을 하나씩 적어야 해서 `잊혀졌다`, `보여집니다`, `닫혀진`을 전부 놓쳤습니다(탐침 11/11 미탐). 📎 `morph_rules.py` 설명

Q65. 문체 일관성은 어떻게 판정하나요?

🟢 문장마다 종결어미로 문체를 분류하고 **다수 문체가 60% 이상**이면 소수 문체 문장을 지적합니다. 반말↔존댓말일 때만 고쳐 쓰기를 제안합니다. 📎 `consistency.py`

Q66. 띄어쓰기는 어떻게 검사하나요?

🟢 네 갈래: ① BiGRU 태거(점선) ② Kiwi가 지운 공백(medium) ③④ 규칙 30여 개(high). 📎 책 10장

Q67. 띄어쓰기 자동 교정은 모델이 하나요?

🟢 **아니요.** 모델 단독으로는 점선(low)까지만. 자동 교정은 **규칙이 정한 자리**에서만 합니다. 📎 `MODEL_NEVER_AUTOFIXES = True`

Q68. 왜 띄어쓰기에서 모델을 믿지 않나요?

🟢 태거는 경계 하나하나 F1 0.945지만 **문장 전체를 다 맞히는 비율은 44.8%**라서 그대로 자동 교정하면 안 됩니다. 📎 `spacing_tagger.py`

Q69. 띄어쓰기 "허용"이 뭔가요?

🟢 규범이 둘 다 허용하는 표기(보조용언 붙여쓰기 등)는 **침묵**합니다. 필수 / 권장 / 허용 3단계, 애매하면 허용으로 보냅니다. 📎 `spacing_tiers.py`

Q70. `못하다`는 왜 안 고치나요?

🟢 한 단어로 굳은 말이고, 위키 13건이 **전부 정문**이었습니다. 📎 `_NEG_LEXICALIZED`

Q71. 오타(`매읽`)는 어떻게 잡나요?

🟢 사전에 없고 Kiwi도 못 읽은 2~3음절 어절을, **키 한 번 실수**로 닿는 흔한 말(`매일`)로 바꿉니다. 대부분 점선입니다. 📎 책 9장

Q72. 키보드 실수를 어떻게 계산하나요?

🟢 글자를 키 입력열로 풀어(`ㄺ` = `ㄹ` + `ㄱ`) **한 키 더/덜/이웃 키**인지 봅니다. `구촌→구몬`은 키보드 양 끝이라 제외. 📎 `keystrokes.py`

Q73. 이상탐지는 뭔가요?

🟢 사전에 없고 글자 이음이 **한국어답지 않은** 어절에 점선만 긋습니다. 고칠 말은 없습니다. 📎 `anomaly.py`, 임계값 11.25

Q74. 이상탐지는 왜 평균이 아니라 최댓값인가요?

🟢 긴 어절의 이상한 이음매 한 곳을 평균이 희석하기 때문입니다(평균은 오문이 정문보다 낮게 나왔음). 📎 `anomaly.py` 설명

파트 5. 모델·데이터

Q75. 직접 만든 모델이 정확히 뭔가요?

🟢 **결(Gyeol) 모델군**: 혼동쌍 판별 6개(4개 가동), 단어 경계 판별기, 형태소 언어모델(gyeol-lm), 어절 사전. 띄어쓰기 BiGRU 태거는 별도입니다. 📎 책 7-9장, NOTICE

Q76. "직접 학습"의 범위는?

🟢 데이터 수집 → 문제·정답 생성 → 학습 → 문턱 선택 → 배포 파일 변환이 **전부 이 저장소 코드**입니다. 빌린 것은 학습 알고리즘(scikit-learn)과 코퍼스 원문뿐입니다. 📎 NOTICE

Q77. 사전학습 모델(BERT 등)을 안 쓴 이유는?

🟢 "확실하게 작동"이 목표였고, 실제로 시험한 외부 교정 모델(KoBART 기반)이 **규칙 시스템보다 성능이 낮고 새 오류를 만들어** 통합하지 않았다는 기록이 있습니다. 📎 CONTEXT.md(과거 기록) ⚠️ 일부 비교 실험 코드(`train_spacing_bert.py`, klue/roberta-base)는 있으나 **배포물에 포함되지 않습니다**.

Q78. 모델이 정확히 어떻게 판단하나요?

🟢 헷갈리는 자리를 ▒로 가리고 **앞뒤 12글자**를 봅니다. 글자 2~4조각마다 학습된 가중치를 더해 확률로 바꿉니다(로지스틱 회귀). 📎 책 7.5 (손계산이 모델 출력과 일치)

Q79. 딥러닝인가요?

🟢 **혼동쌍·경계 모델은 아닙니다**(글자 조각 + 로지스틱 회귀, 선형 모델). 띄어쓰기 태거만 BiGRU 신경망입니다. 📎 `train.py`, `spacing_model_nn.py`

Q80. 왜 선형 모델인가요?

🟢 작고(모델당 약 5.7MB), 빠르고, **판단 이유(어느 조각이 밀었나)를 손으로 계산해 설명할 수 있기 때문**입니다. 📎 책 7.5

Q81. 학습 데이터에 오류 문장이 필요한가요?

🟢 **필요 없습니다.** 맞는 글에서 슬롯을 가리면 원문 글자가 곧 정답입니다. 문제와 정답이 같이 생깁니다. 📎 책 7.3

Q82. 데이터는 어디서 나왔나요?

🟢 **한국어 위키백과**(CC BY-SA 4.0)와 **nsmc 영화평**(CC0). 📎 NOTICE

Q83. 위키는 격식체인데 구어는요?

🟢 약점입니다. 구어 시드 문장을 손으로 보탰고, 언어모델에는 구어를 8배 가중해 넣었습니다. `데/대` · `러/려`는 구어 표본이 없어 **꺼 놓았습니다**. 📎 `COLLOQUIAL_SEED`, `train_ngram_lm.py`

Q84. 과적합은 어떻게 막나요?

🟢 시험용을 **문장 단위**로 분리(같은 문장의 슬롯이 양쪽에 안 들어가게), 평가 문장은 학습에서 제외, 평가셋도 세 개로 나눠 처음 보는 문장을 따로 봅니다. 📎 책 7.7, 8.9

Q85. 확률 문턱은 누가 정하나요?

🟢 **측정이 정합니다.** 정문 오탐률 ≤0.5%를 지키는 가장 낮은 문턱을 자동 선택하고, 못 맞추면 모델을 싣지 않습니다. 📎 `train.py`

Q86. 문턱이 모델마다 다른가요?

🟢 네. an 0.5 · doe 0.6 · den 0.7 · roseo 0.7 · (꺼 둔) reo/dae 0.8 · 경계 0.97. 📎 `meta.json`, `boundary_meta.json`

Q87. 모델이 확신해도 틀릴 수 있잖아요?

🟢 그래서 **문법으로 답이 정해지는 자리는 규칙이 먼저** 확정하고(`되` + `고` 등), 모델이 자신 있게 틀리는 부류(이름 속 `든`, 연결어미 `-되`)는 품사로 **잠급니다**. 📎 책 7.9 게이트 표

Q88. 로서/로써가 약한 이유는?

🟢 수단/자격은 **의미** 차이라 글자 창으로는 잘 안 보입니다. 모델 단독 재현율 20%, 규칙 포함 엔진 재현율 약 85%. 📎 `meta.json`(0.203), pipeline(0.8503)

Q89. 경계 모델은 뭘 하나요?

🟢 Kiwi가 "여기 띄우자"고 한 자리마다 **진짜 단어 경계인지** 판정합니다. 새 경계를 스스로 찾지는 못합니다. 📎 책 8장

Q90. 경계 모델 성능이 낮아 보이는데요?

🟢 문턱 0.97에서 진짜 경계의 **9.3%만** 모델 혼자 고칩니다. 대신 정문 문장을 잘못 끊는 비율이 0.03%입니다. **"확실한 것만 잡는" 설계**입니다. 📎 `boundary_meta.json`

Q91. 데이터를 더 늘리면 좋아지지 않나요?

🟢 5번 재학습으로 확인했는데 **제자리**였습니다. 데이터가 많을수록 오탐 추정이 정확해져 문턱이 올라가고, 얻은 만큼 잃습니다. 한계는 모델 종류에 있다고 기록돼 있습니다. 📎 `train_boundary.py` 설명

Q92. 언어모델(gyeol-lm)은 뭔가요?

🟢 형태소 3개씩 묶어 센 표로, 두 표기 중 **어느 쪽이 더 한국어다운지 점수**를 냅니다. 고칠 말은 **만들지 않고 순위만** 매깁니다. 📎 책 9.5

Q93. 언어모델이 후보를 만들지 못하게 한 이유?

🟢 만들면 엔진이 말하는 범위가 넓어져, 새 오탐이 **측정 밖에서** 생기기 때문입니다. 📎 `rescore.py` 설명

Q94. 어절 사전은 뭔가요?

🟢 위키에서 어절별 등장 횟수를 센 표(**1,806,050개**, 21.7MB). "이 말이 있는가"와 "얼마나 흔한가"를 답합니다. 📎 책 9.1

Q95. 사전에 단어 대신 해시를 저장하는 이유는?

🟢 메모리 때문입니다. 해시 충돌이 나면 "모르는 말을 안다고 착각"하는 쪽(놓침)으로 틀려서 **안전한 방향**입니다. 📎 `lexicon.py`

Q96. `.ddm`은 뭔가요?

🟢 다듬이 정한 모델 파일 형식입니다. sklearn 없이 읽혀서 메모리가 117MB → 5.7MB, 변환 때 원본과 확률을 대조(최대 오차 5.55e-16)합니다. 📎 책 7.8

Q97. 모델 파일 크기는?

🟢 혼동쌍 `.ddm` 각 2~6MB, 경계 5.7MB, 언어모델 36MB, 어절 사전 22MB. 📎 `ls ml/models`

Q98. BiGRU 태거는 왜 배포에서 빠졌나요?

🟢 torch(딥러닝 라이브러리)가 필요한데 배포 이미지에 설치하지 않기 때문입니다. 지표 기여도(`l1_recall`)는 0.0012로 미미합니다. 📎 README 한계 표

Q99. 학습을 직접 해 볼 수 있나요?

🟢 `python -m ml.generate_data fetch → build`, `python -m ml.train`, `python -m ml.export_models`. 📎 `ml/` 각 파일 상단 사용법

Q100. 학습형 교정(문장 통째 고치기)은 시도했나요?

🟢 7/24에 시도했고 **안전선(오탐 하한)을 못 넘어** 접었습니다. 이후 "좁은 판정을 정확히" 방향으로 갔습니다. 📎 커밋 `Per-edit-type gating: decisive negative result redirects the strategy`, docs/PLAN-learned-gec.md

파트 6. 평가·수치

> 수치는 2026-10-11에 제가 `python -m eval.pipeline` 과 `eval.real_errors.score` 를 **직접 돌려** 얻은 값입니다. README 값과 다른 곳은 표시했습니다.

Q101. 정확도가 몇 %인가요?

🟢 질문에 따라 답이 다릅니다. **고친 것 중 맞은 비율(정밀도) 99.2%, 사람이 쓴 오류를 자동으로 고치는 비율 약 67~74%.** ⚠️ "정확도 99%"만 말하면 과장입니다. 반드시 둘을 같이 말하세요.

Q102. 정밀도가 뭔가요?

🟢 엔진이 "고쳐라"고 한 것 중 **실제로 맞게 고친** 비율. 측정값 0.9921(하한 0.97).

Q103. 재현율은요?

🟢 실제 오류 중 **잡아낸** 비율. 합성 오류 기준 규칙층 97.9%, 혼동쌍 99.2%. **실사용 오류 기준은 Q105.**

Q104. 정문 오탐률은?

🟢 **0.00%**(하한 ≤0.2%). 맞는 글 9,765문장에서 건드린 비율입니다.

Q105. 실제 사람이 쓴 오류는 얼마나 잡나요?

🟢 자동교정 기준 **dev 74% · holdout 72% · holdout2(처음 본 문장) 67%**, 점선 포함 81/75/72%. ⚠️ README는 holdout2를 63%로 적고 있습니다. 제 측정은 67%이고 차이의 원인은 확인하지 못했습니다. 발표에서는 **"60%대 중후반"**이라 하세요.

Q106. 왜 세 개로 나눠 쓰나요?

🟢 dev는 규칙을 만들 때 계속 보는 셋(과적합), holdout은 규칙 만들기 전에 봉인했지만 이후 여러 번 봄, holdout2는 **처음 보는 문장**. 차이가 곧 과적합의 크기입니다. 📎 README

Q107. 합성 오류 점수와 실사용 점수가 왜 다른가요?

🟢 합성 오류는 **규칙이 넣은 오류를 규칙이 잡는** 구조라 순환입니다. 같은 엔진이 합성 93.8%, 실사용 약 15%(당시)였습니다. 📎 `score.py` 설명

Q108. 평가에 쓴 데이터는?

🟢 정문 9,765(위키) · 합성 오류 3,871 · 규범 예문 225 · 구어 정문 3,000/오류 1,073(nsmc) · 실사용 오류 dev 300/holdout 199/holdout2 150문장. 📎 README "평가 데이터"

Q109. 실사용 오류는 누가 표시했나요?

🟢 **주석자 한 명(Claude)**, 한글 맞춤법 기준. 경계 사례가 섞여 있어 **절대값보다 변화량**을 봅니다. ⚠️ 주석자가 1명이라는 점이 약점입니다.

Q110. 어떤 부류를 잘 잡고 못 잡나요?

🟢 (dev) 의존명사 94% · 시간명사 88% · 관형사 86% / **기타 34% · 구어 표기 38%**. 조사는 점선 포함 86%. 📎 `score.py dev` 출력

Q111. 구어(카톡·댓글)는요?

🟢 재현율 **95.2%**입니다. 구어 **정밀도는 재지 않습니다** — 구어 "정문"에도 실제 띄어쓰기 오류가 많아 코퍼스를 재게 되기 때문입니다. 📎 pipeline `[casual]`

Q112. 구어에서 지적이 너무 많지 않나요?

🟢 두 문장 반에 하나꼴(0.397, 하한 ≤0.40)입니다. 대부분 원문의 진짜 오류지만 많다고 느낄 수 있고, **낮출지는 제품 결정**이라고 문서에 적혀 있습니다. 📎 pipeline, README

Q113. 품질을 어떻게 보장하나요?

🟢 한 명령 `python -m eval.pipeline`이 **단위 테스트 → 모델·사전 대조 → QA 불변식 → 메트릭 하한**을 차례로 돌립니다. 하나라도 실패하면 막힙니다. 📎 책 11장

Q114. 테스트는 몇 개?

🟢 제 실행에서 **1,020개 통과**(37.6초). README는 997개로 적혀 있습니다(이후 늘어난 것으로 보임).

Q115. "하한(floor)"이 뭔가요?

🟢 **후퇴 금지선**. 정밀도 ≥0.97, 오탐률 ≤0.002, 과교정률 ≤0.002, 규칙층 재현율 ≥0.97, 혼동쌍 ≥0.99, 로서/로써 ≥0.82. 통과시키려고 **내리지 않습니다.**

Q116. 하한이 목표인가요?

🟢 아니요. "여기서 더 나빠지면 안 된다"일 뿐 "충분하다"가 아닙니다. 📎 DESIGN 1.8

Q117. QA 불변식이란?

🟢 어떤 입력에서도 참이어야 하는 성질: 고쳐서 나빠지지 않음(P1), 두 번 고쳐도 같음(P2), 정문에서 high 오탐 0(P3), 고침 폭 최소(P4), 문서 속에서도 같은 결과(P5). 📎 `properties.py`

Q118. 회귀 레지스트리는요?

🟢 한 번 틀리게 고쳤던 문장을 **영구 등록**해 다시는 안 틀리게 막는 목록(현재 22건, 실패 0).

Q119. 퍼즈 결과는?

🟢 정문에 무작위 오류를 넣어 검출률을 봅니다. 안/않 100%, 되/돼 94%, 조사 바꾸기 58%, 띄어쓰기 추가 35%, **글자 빠짐 4.6%·글자 중복 3.2%·키보드 7.2%**. ⚠️ 낮은 수치를 숨기지 않습니다. 글자 하나 빠진 오타는 단어를 알아야 잡습니다.

Q120. 규칙별 오탐률은 왜 재나요?

🟢 전체 오탐률은 규칙 수백 개를 한 숫자로 뭉쳐 결함을 **희석**합니다. `den`은 전체 0.0009일 때 자기 표본에서 0.133이었습니다(148배). 📎 책 11.4

Q121. 표본이 적으면?

🟢 20건 미만이면 **숫자를 내지 않고 "못 쟀다"**로 표시합니다. 0%와 못 쟀다는 다릅니다.

Q122. 측정 도구에 버그가 있었나요?

🟢 있었습니다. 공백을 지우는 교정(제안 `""`)을 자동교정으로 세지 않아 붙여쓰기 부류가 29%(실제 79%)로 찍혔고 정밀도도 실제보다 좋게 보였습니다. 고친 뒤 숫자가 나빠졌고 하한은 내리지 않았습니다. 📎 `score.py` 주석, README

Q123. CI는 돌고 있나요?

🟢 GitHub Actions `quality-pipeline`이 있습니다. 과거 **한 달간 안 돈 적**이 있었고(트리거 브랜치 불일치) 8/30에 고쳤습니다. 📎 DESIGN 2.4 ⚠️ 현재 CI 상태는 확인하지 않았습니다.

Q124. 속도 측정은?

🟢 문서 기준 짧은 문장 ~2ms, 6,400자 ~0.5초(README). 제가 잰 값은 콜드 스타트 첫 요청 4.3초, 이후 수 ms. 📎 README 배지, DEPLOY.md

파트 7. 배포·보안·라이선스

Q125. 서버 사양은?

🟢 워커당 **약 600MB RAM**(거의 전부 Kiwi). 최소 **1GB** 인스턴스, `WEB_CONCURRENCY=1` 권장. 📎 DEPLOY.md 4절 (2026-09-14 실측)

Q126. 도커로 배포되나요?

🟢 네. `docker compose up -d --build` → `http://localhost:8000`. gunicorn+uvicorn, `/api/health` 헬스체크. 📎 DEPLOY.md 3절

Q127. 이미지 크기는?

🟢 프로덕션 의존성 약 250MB(`requirements.lock`). 학습용 sklearn/scipy는 `requirements-dev.txt`로 분리.

Q128. 환경 변수는?

🟢 `ALLOWED_ORIGINS`(CORS), `RATE_LIMIT_CHECK`(분당 60), `TRUSTED_PROXIES`, `WEB_CONCURRENCY`, `KIWI_DEFAULT_DICT`, `KIWI_MULTI_DICT`. 📎 DEPLOY.md 2절

Q129. 많은 요청이 오면?

🟢 IP별 **분당 60회** 제한(429 + 재시도 시간). 한 대 기준이며 다중 인스턴스는 Redis 등으로 바꿔야 한다고 코드에 적혀 있습니다. 📎 `ratelimit.py`

Q130. IP 위조는 막나요?

🟢 `X-Forwarded-For`를 **기본으로 믿지 않습니다.** 신뢰 프록시(`TRUSTED_PROXIES`)에서 온 것만 인정합니다. 📎 `clientip.py`

Q131. 개인정보는?

🟢 글이 **기기 밖으로 안 나가고**, 맥 앱이 읽은 글은 디스크에 남기지 않으며, 저장되는 것은 "항상 허용" 목록뿐입니다. 📎 SECURITY.md

Q132. 비밀번호 칸은 읽나요?

🟢 **읽지 않습니다.** 맥 앱은 `AXSecureTextField`와 보안 입력 모드를 제외하고, 확장은 비밀번호 필드를 **옵트인**으로만 검사합니다. 📎 `main.swift`, CONTEXT.md

Q133. 맥 앱은 어떤 권한을 쓰나요?

🟢 **손쉬운 사용(Accessibility)** 권한으로 다른 앱의 입력창을 읽고, 교정을 적용합니다. 📎 SECURITY.md

Q134. 맥 앱이 다른 앱의 데이터를 덮어쓰면?

🟢 쓰기 전에 **그 자리에 원문이 정말 있는지** 확인하고, 결과가 기대와 같은지 다시 확인합니다. SECURITY.md가 이 경로를 "심각"으로 분류합니다. 📎 `applyCorrection`

Q135. 서버가 외부에 노출될 위험은?

🟢 `localhost` 기준 설계입니다. 외부 노출 경로를 SECURITY.md가 "심각"으로 다룹니다. CORS 기본이 `*`이므로 **외부에 올릴 때는 `ALLOWED_ORIGINS`를 지정**해야 합니다. 📎 `main.py`, DEPLOY.md

Q136. 취약점은 어디로 제보?

🟢 공개 이슈 말고 저장소 소유자에게 직접. 최신 master만 고칩니다. 📎 SECURITY.md

Q137. 라이선스 문제는 없나요?

🟢 코드 MIT, 데이터는 위키 CC BY-SA 4.0·nsmc CC0, 모델은 위키 파생물이라 **CC BY-SA 조건을 따라 표기**했습니다. 📎 NOTICE

Q138. Kiwi 라이선스는?

🟢 kiwipiepy 0.23.2는 **LGPL-3.0**입니다(패키지 메타데이터 확인). ⚠️ NOTICE에 Kiwi 항목이 **없습니다.** 배포 전에 보완이 필요하며, 법적 판단은 제가 하지 않았습니다.

Q139. 국립국어원 데이터를 쓰나요?

🟢 사전 규칙의 **근거는 맞춤법 규범**(국립국어원)이지만, 국립국어원 코퍼스를 학습에 쓰지는 않았습니다. 평가셋 `gold`는 규범 예문을 정리한 것입니다. 📎 README, 메모(코퍼스 라이선스) ⚠️ AI Hub 데이터는 사용하지 않기로 했다는 기록만 있고 사유는 저장소에 명시돼 있지 않습니다.

Q140. 크롬 웹스토어에 올라가 있나요?

🟢 이 자료에는 **없습니다.** 확장은 개발자 모드로 로드하는 방식이고, 스토어 배포는 DEPLOY.md가 "범위 밖"이라고 적었습니다.

Q141. 확장이 실제로 되나요?

🟢 README는 "2026-09-16 밑줄 미표시·원인 미규명"이라 적었고, 9/13 헤드리스 Chromium에서는 정상이었습니다. **실제 Chrome에서의 재확인이 필요하다**고 DESIGN 3부에 있습니다. ⚠️ 발표 시연 전에 **직접 확인**하세요. 저는 이 대화에서 확장을 실행하지 못했습니다.

Q142. 윈도우는?

🟢 이 자료에는 **윈도우용 앱이 없습니다.** 맥 앱은 macOS 전용이고, 서버는 파이썬이라 돌아가는 곳에서 웹 앱을 쓸 수 있습니다. ⚠️ 윈도우에서 서버를 실제로 돌려 본 기록은 확인하지 못했습니다.

파트 8. 개발 과정·팀

Q143. 얼마나 걸렸나요?

🟢 첫 커밋 **2026-07-14**, 마지막 **2026-09-21**. 약 10주, 커밋 377개. 📎 `git log`

Q144. 개발이 어떤 순서로 진행됐나요?

🟢 **넓게 만들기(7월) → 재어 보기(8월 말) → 걷어내기(9/15) → 좁게 깊게(9월)**. 📎 책 13.1 타임라인

Q145. 가장 큰 방향 전환은?

🟢 ① 학습형 교정 접음(7/24) ② 상용 서비스·AI 층 제거(9/15) ③ 합성 점수 → **실사용 오류 기준**으로 전환(9/13). 📎 책 13.2

Q146. 왜 AI 층과 결제를 걷어냈나요?

🟢 정체성(완전 로컬·무과금·개인정보)과 맞지 않고, AI의 독자 추측은 **통제 못 하는 오탐 원천**이기 때문으로 보입니다. ⚠️ 이것은 문서·코드 맥락에서의 **해석**입니다. 커밋 메시지에 이유가 전부 적혀 있지는 않습니다.

Q147. 처음부터 로컬이었나요?

🟢 아니요. 처음엔 규칙 기반 + AI 층(Gemini)이었고 9/15에 지금 구조(규칙+모델, 완전 로컬)가 됐습니다.

Q148. 실패 사례를 하나만 든다면?

🟢 **연결어미 `-되` (적되·하되)를 `적돼` 로 고치던 과교정**입니다. 정문 10,000문장에 그 구문이 1건뿐이라 지표가 안 움직여서 못 봤고, 이후 **커버리지 게이트**가 생겼습니다. 📎 책 13.4 ②

Q149. 그 실패에서 무엇을 배웠나요?

🟢 "지표가 좋아도 **그 구문을 재고는 있는지** 먼저 봐야 한다." 그래서 지표보다 커버리지를 먼저 찍습니다.

Q150. 가장 의외였던 발견은?

🟢 합성 평가 93.8% 엔진이 실제 사람의 오류는 약 15%밖에 못 고친다는 것. 이후 개선 기준이 바뀌었습니다.

Q151. 코드 중 사람이 쓴 부분과 AI가 쓴 부분은?

🟢 git 기록상 zmflm0110 커밋 253개 중 239개에 `Co-Authored-By: Claude`가 있어 **Claude Code 세션 작성**입니다. byeolki 커밋 124개에는 표시가 없습니다. ⚠️ 사람이 정한 것은 방향(오탐 금지, 실사용 기준, AI 층 제거 등)이라고 프로젝트 기록에 있습니다. **누가 한 줄씩 썼는지는 git만으로 단정할 수 없습니다.**

Q152. 팀은 몇 명인가요?

🟢 git 작성자는 **두 계정**입니다(zmflm0110, byeolki). 실제 사람 수는 이 자료로 알 수 없습니다.

Q153. 두 사람이 각자 한 일은?

🟢 **zmflm0110**: 초기 구축·평가 체계·맥 앱·확장·공유 코어·실사용 평가셋. **byeolki**: 걷어내기·구조 정리·문법 규칙 다수·경계 모델·어절 사전·언어모델·개인 사전·디자인 통합. 📎 `git log`, 책 대화 기록

Q154. 앞으로 계획은?

🟢 기준은 **처음 보는 문장에서의 재현율**입니다. 경계 모델은 모델 종류의 한계가 확인돼 "다음 종류의 모델"이 필요하다고 코드에 적혀 있습니다. 📎 `train_boundary.py`, PLAN-real-errors.md

Q155. 기여하려면?

🟢 `CONTRIBUTING.md`의 절차: 부류 찾기 → 형태소 조건 → **정문에 걸어 발동을 한 줄씩 읽기** → 게이트 → 테스트 고정. **하한을 내리지 않습니다.** 📎 CONTRIBUTING.md

Q156. 제보만 하고 싶다면?

🟢 이슈 템플릿으로 **과교정 신고**나 **못 잡는 오류**를 남기면 됩니다. 과교정 신고는 그대로 회귀 테스트가 됩니다.

파트 9. 곤란한 질문

Q157. 결국 Kiwi 쓰는 거 아닌가요? 직접 만든 게 맞나요?

🟢 **Kiwi는 형태소 분석을 맡은 외부 라이브러리가 맞습니다.** 그 위에서 규칙 21개 검사기와 직접 학습한 모델들이 돕니다. "형태소 분석은 Kiwi, 교정 판단은 직접"이라고 구분해 말하세요.

Q158. 99%라더니 사람 오류는 67%잖아요. 과장 아닌가요?

🟢 두 숫자는 **다른 것**을 잽니다. 99%는 "고친 것의 정확도", 67%는 "오류를 놓치지 않은 비율"입니다. 이 프로젝트는 일부러 **놓치는 쪽을 택해** 오탐을 0으로 둡니다. 두 숫자를 같이 공개하는 것이 이 프로젝트의 태도입니다.

Q159. 나머지 33%는 왜 못 잡나요?

🟢 ① 규칙이 없는 부류(기타 34%) ② 의미로만 갈리는 것 ③ 맞춤법이 허용하는 표기라 **일부러 침묵**. 그리고 점선으로는 5%p 더 표시합니다.

Q160. ChatGPT에 물어보면 되지 않나요?

🟢 다듬의 가치는 **로컬·즉시(수 ms)·일관·오탐 통제·근거(규정 조항) 제시**입니다. 외부 AI는 같은 입력에 다른 답을 낼 수 있고 글이 외부로 나갑니다. ⚠️ 성능 비교 수치는 이 자료에 없으니 "더 정확하다"고 말하지 마세요.

Q161. 규칙이 474개면 수작업 아닌가요? 확장성이 없지 않나요?

🟢 맞습니다. 규칙은 수작업이고, 그래서 **규칙 하나를 넣는 절차**(정문에 걸기·한 줄씩 읽기)가 있습니다. 규칙으로 못 가르는 부류(경계·혼동)는 모델로 옮겼습니다.

Q162. 모델이 선형이라 약하지 않나요?

🟢 판단 이유를 설명할 수 있고 가볍습니다. **한계는 문서에 실측으로 적혀 있습니다**(경계 모델은 데이터를 늘려도 제자리, 모델 종류의 한계). 더 큰 모델을 쓰려면 오탐 통제와 로컬 배포 부담이 따릅니다.

Q163. 평가를 자기가 만든 데이터로 했다면 믿을 수 있나요?

🟢 약점입니다. 그래서 ① 규칙을 만들기 전에 봉인한 holdout ② 처음 보는 holdout2 ③ 주석 기준 공개를 두었습니다. 그러나 **주석자가 한 명(Claude)**이고 표본이 약 500문장이라는 한계는 문서에도 있습니다.

Q164. 실제 사용자 테스트는 했나요?

🟢 이 자료에는 **사용자 연구 기록이 없습니다.** 개발 중 사용자가 준 문장·피드백으로 규칙을 고친 기록은 있습니다(CONTEXT.md). ⚠️ "사용자 N명이 썼다" 같은 말은 하지 마세요.

Q165. 맥에서만 되는 건 한계 아닌가요?

🟢 네. 데스크톱 앱은 macOS 접근성 API에 의존합니다. 웹·확장은 플랫폼과 무관합니다.

Q166. 확장이 안 된다는 문서가 있던데요?

🟢 README에 "2026-09-16 밑줄을 그리지 않는다"가 있고, 9/13 헤드리스 실측에선 정상이었습니다. **"재현이 안 된 것이지 고친 게 아니다"**라고 DESIGN이 적었습니다. 시연 전 직접 확인이 필요합니다.

Q167. 테스트가 1,020개면 충분한가요?

🟢 개수보다 **구조**가 중요합니다: 단위 테스트 + 정문 침묵 테스트 + QA 불변식 + 회귀 레지스트리 + 규칙별 오탐률 + 커버리지 게이트.

Q168. 문서가 코드와 안 맞는 곳이 있던데요?

🟢 있습니다. README의 사전 규칙 수(436 vs 474), 타이핑 대기(1.5초 vs 0.7초), 펫 설명(제거됨), holdout2(63 vs 67%), CONTEXT.md의 Gemini 서술(제거 전 구조). **코드가 최신**입니다. 📎 책 13.6

Q169. 개인 사전에 넣으면 오류가 영영 안 보이나요?

🟢 그 단어(접두 일치)만 안 보입니다. 사전 칩의 ×로 빼면 다음 검사부터 다시 지적됩니다.
📎 `forgetWord`

Q170. 오탐이 0%라는 게 말이 되나요?

🟢 **측정한 정문 9,765문장에서** 0건입니다. 검토 도중 정문에서 발동한 것은 원문 오류로 확인된 문장을 **오염 목록으로 옮겼다**는 점도 같이 말해야 합니다(그래서 정문 수가 줄어듦). ⚠️ "모든 글에서 오탐이 없다"는 뜻이 아닙니다.

Q171. 오염 목록으로 빼면 성적 조작 아닌가요?

🟢 근거를 적어 **한 줄씩 읽고** 원문 자체가 틀린 문장만 뺍니다(위키 원문 오류율 약 11.7%라는 실측). 규칙이 틀린 자리는 **빼지 않고 규칙을 좁힙니다.** 목록은 파일로 공개돼 있습니다(`*_manual_review.jsonl`).

Q172. 구어 오탐률 38%는 뭔가요?

🟢 코퍼스의 "정문"에 진짜 띄어쓰기 오류가 많아 **엔진이 아니라 코퍼스를 재는 수치**입니다. 무작위 20건 중 19건이 원문의 진짜 오류였습니다. 그래서 하한은 걸지 않고 "지적 빈도"만 관리합니다.

Q173. 하한을 몰래 내린 적은?

🟢 문서상 **없다**고 적혀 있습니다("하한은 하나도 내리지 않았습니다"). 단 구어 지적률 상한을 .34→.40으로 **올린 적은** 있고, 이유(코퍼스 오염)를 코드 주석에 길게 적었습니다. 📎 `pipeline.py` CASUAL_FLOORS 주석

Q174. 한국어 외 언어로 확장 가능한가요?

🟢 이 자료에는 계획이 없습니다. 구조(슬롯 판별 + 규칙 + 병합)는 일반적이지만 규칙·사전은 한국어 전용입니다.

Q175. 비즈니스 모델은?

🟢 7월에 과금 구조를 만들었다가 9/15에 제거했고 현재는 **없습니다.** MIT 라이선스 오픈소스 도구입니다.

Q176. 가장 자랑할 점 하나만?

🟢 **"측정이 틀렸을 때 그것을 찾아내는 장치"**: 커버리지 게이트, 규칙별 오탐률, 실사용 평가셋. 숫자가 나빠져도 정직하게 공개하는 방식입니다.

파트 10. 숫자 치트시트 (1장)

규모

항목	값
개발 기간	2026-07-14 ~ 09-21 (약 10주)
커밋	377개 (두 계정)
검사기 / 출처	21개 / 58개
사전 규칙	474개
단위 테스트	1,020개 통과
어절 사전	1,806,050개
평가 문장	정문 9,765 · 합성 오류 3,871 · 실사용 dev 300 / holdout 199 / holdout2 150

성능 (2026-10-11 직접 실측)

항목	값	한 줄 해석
정밀도	**99.21%**	고친 것 중 맞은 비율
정문 오탐·과교정	**0.00%**	맞는 글을 안 건드림
규칙층 재현율	97.9%	합성 오류
혼동쌍 재현율	99.2%	합성 오류
로서/로써	85.0%	가장 약한 쌍
실사용 자동교정	74 / 72 / **67%**	dev / holdout / holdout2
점선 포함	81 / 75 / 72%	
구어 재현율	95.2%	
구어 지적 빈도	0.397	두 문장 반에 하나
퍼즈 약점	글자 빠짐 4.6% · 중복 3.2%	단어를 알아야 잡음

모델

모델	문턱	상태
안/않	0.5	가동
되/돼	0.6	가동
던/든	0.7	가동
로서/로써	0.7	가동 (모델 단독 재현율 20%)
러/려 · 데/대	0.8	학습됨, **꺼 둠** (평가 표본 없음)
경계 판별기	0.97	가동 (진짜 경계의 9.3%만 모델이 고침)

한 줄 답변 모음 (외워 두기)

1. **무엇?** 틀리게 고치는 것을 가장 무겁게 보는 완전 로컬 한국어 교정기.
2. **어떻게?** 형태소 분석 → 검사기 21개가 후보 제출 → 우선순위로 정리.
3. **AI?** 외부 AI 없음. 직접 학습한 작은 판별 모델들 + 규칙.
4. **정확도?** 고친 것의 99% 맞음, 정문 건드리지 않음. 사람 오류는 처음 본 문장 기준 약 67%를 자동 교정.
5. **왜 100%가 아니냐?** 확신 없으면 고치지 않고 점선으로 표시하기 때문. 오탐 > 미탐.
6. **약점?** 의미로 갈리는 쌍, 글자 빠짐 오타, 주석자 1명, 확장의 실제 Chrome 재확인.
7. **독창성?** 오탐 통제 설계 + 측정이 틀렸을 때 찾아내는 평가 장치.
8. **개인정보?** 글이 기기 밖으로 안 나감. 비밀번호 칸 제외.

말하면 안 되는 것 (이 자료로 뒷받침되지 않음)

- 다른 제품과의 성능 **비교 수치**
- “사용자 N명이 사용” 같은 **사용자 규모**
- “100% 정확” / “오탐 전혀 없음”(측정한 9,765문장 기준에서만 0)
- “모든 것을 직접 만들었다”(Kiwi는 외부)
- “크롬 확장이 완벽히 동작한다”(실제 Chrome 재확인 필요)